MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES

(Deemed to be University)
SCHOOL OF COMPUTER APPLICATIONS

PRACTICAL ASSIGNMENT REPORT

DATA STRUCTURES USING PYTHON

Course / Program:	Master of Computer Applications (MCA)
Semester:	1st Semester
Subject:	Data Structures Using Python
Submitted By:	Sumit Mishra
Roll Number / UID:	26MCAP101N030
Academic Session:	2026–2027

Submitted To:

Faculty Name: Sachin Sharma Sir

Table of Contents (Index)

Program 1	Insertion and Deletion in an Array	3
Program 2	General Queue Operations	4
Program 3	Circular Queue Operations	5
Program 4	Sum of Diagonal and Non-Diagonal Elements of a Matrix	6
Program 5	Product of Two Matrices	7
Program 6	Stack Operations (Push, Pop and Display)	8
Program 7	Reverse of a String Using Stack	9
Program 8	Matching Parentheses Using Stack	10
Program 9	Electricity Bill Calculation	11
Program 10	Check Even or Odd Number	12
Program 11	Find Largest and Smallest of Three Numbers	13
Program 12	Student Grade According to Marks	14
Summary	Assignment Summary & Key Learnings	15
Thank You	Conclusion & Acknowledgement	16



PROGRAM 1 - : Check the Electricity bill as per unit and gave the amount to the user.

Sumit_Ds_lab > Electricity_bill.py > ...

```python
# Electricity Bill
print("------------Electricity Bill -------")

#Take the electricity units consumed from the user
unit = int(input("Enter electricity units: "))

if unit <= 100:
    bill = unit * 3
elif unit > 101 and unit <= 300:
    bill = unit * 4
else:
    bill = unit * 5

print("Electricity bill =", bill)
print("------------------------------------------------------")
print()

```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Electricity_bill.py
------------Electricity Bill -------
Enter electricity units: 398
Electricity bill = 1990
------------------------------------------------------

PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 2 : - Check the number is even or odd

Sumit_Ds_lab > Check_Even_odd.py > ...

```python
#Check even or odd number
print("------------Check even or odd number -------")
n = int(input("Enter a number: "))

if n % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")
print("------------------------------------------------------")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Check_Even_odd.py
------------Check even or odd number -------
Enter a number: 78
Number is Even
-----------------------------------------------------
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 3 : - Find the largest and smallest of three numbers.

Sumit_Ds_lab > Find_largest_Number.py > ...

```python
#Find Largest and Smallest of 3 Numbers

print("------------Find Largest and Smallest of 3 Numbers -------")
#Take 3 numbers from the user
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a > b and a > c:
    largest = a
elif b > c:
    largest = b
else:
    largest = c
if a < b and a < c:
    smallest = a
elif b < c:
    smallest = b
else:
    smallest = c
print("Largest number:", largest)
print("Smallest number:", smallest)
print("-----------------------------------------------------")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Find_largest_Number.py
------------Find Largest and Smallest of 3 Numbers -------
Enter first number: 7
Enter second number: 9
Enter third number: 3
Largest number: 9
Smallest number: 3
-----------------------------------------------------
```

Program 4 : - Check the marks and gave the grade to students .

```
print("------------Grade of Student According to Marks -------")
#Take the marks of the student
marks = int(input("Enter student marks: "))

if marks >= 90 and marks <= 100:
    print("Excellent")
    print("Student has scored A+ grade .")
elif marks >= 70 and marks < 90:
    print("Very Good")
    print("Student has scored A grade .")
elif marks >= 60 and marks < 70:
    print("Good")
    print("Student has scored B grade .")
elif marks >= 50 and marks < 60:
    print("Fair")
    print("Student has scored C grade . ")
elif marks >= 0 and marks < 50:
    print("Poor")
    print("Student has scored D grade . ")
else:
    print("Invalid marks")
```

Output:-

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Grade_given.py
------------Grade of Student According to Marks -------
Enter student marks: 99
Excellent
Student has scored A+ grade .
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 5 : - Insertion and deletion of array .

Sumit_Ds_lab > Array_Ins_Del.py > ...

```python
# Insertion and deletion of a Array

# For take the array size
print("Insertion and Deletion of an Array")

n = int(input("Enter the array size : "))

#Make the array that is [0,0,0,...,0]
arr = [0]*(n + 1)

#Input value in the array
for i in range(n):
    arr[i] = int(input("Enter the  element : "))

print("Original array:", arr[:n])

print("1. Insertion")
print("2. Deletion")
choice = int(input("Enter your choice : "))

#For insertion choice of array
if choice == 1:
    element = int(input("Enter element to insert : "))
    position = int(input("Enter position in array : "))

    if position < 0 or position > n:
        print("Invalid position , please enter position between 0 and", n)
    else:
        i = n
        while i > position:
            arr[i] = arr[i - 1]
            i = i - 1

        arr[position] = element
        n = n + 1

        n = n + 1

        print("Array after insertion:", arr[:n])
        print('-----------------------------------------------------')

elif choice == 2:
    position = int(input("Enter position to delete : "))

    if position < 0 or position >= n:
        print("Invalid position , please enter position between 0 and", n - 1)
    else:
        i = position
        while i < n - 1:
            arr[i] = arr[i + 1]
            i = i + 1

        n = n - 1
        print("Array after deletion :", arr[:n])
        print('-----------------------------------------------------')

else:
    print("Invalid choice")
```

OUTPUT : -

```
Insertion and Deletion of an Array
Enter the array size : 3
Enter the  element : 43
Enter the  element : 56
Enter the  element : 35
Original array: [43, 56, 35]
1. Insertion
2. Deletion
Enter your choice : 1
Enter element to insert : 49
Enter position in array : 2
Array after insertion: [43, 56, 49, 35]
```

```
Insertion and Deletion of an Array
Enter the array size : 4
Enter the  element : 56
Enter the  element : 45
Enter the  element : 34
Enter the  element : 56
Original array: [56, 45, 34, 56]
1. Insertion
2. Deletion
Enter your choice : 2
Enter position to delete : 3
Array after deletion : [56, 45, 34]
```

Program 7 : - WAP for product of two matrix

Sumit_Ds_lab > Product_two_matrix.py > ...

```python
# Program to multiply two matrices in python
#Input the number for the matrix
print("------------Product of two matrices -------")
#Take the size of the matrix from the user
n = int(input("Enter the matrix size : "))
a = []
b = []
result = []

#Take the first matrix element
print("Enter the elements of first matrix :")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    a.append(row)
    print("First matrix is made successfully . ")

#Take the second matrix element
print("Enter the elements of second matrix : ")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    b.append(row)
    print("Second matrix is made successfully . ")

#Gave the result for the python
for i in range(n):
    row = []
    for j in range(n):
        total = 0
        for k in range(n):
            total = total + a[i][k] * b[k][j]
        row.append(total)
    result.append(row)
```

```
print("Product of matrices :")
for row in result:
    print(row)
print("-----------------------------------------------------------")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Product_two_matrix.py
------------Product of two matrices -------
Enter the matrix size : 2
Enter the elements of first matrix :
3
5
First matrix is made successfully .
6
7
First matrix is made successfully .
Enter the elements of second matrix :
7
8
Second matrix is made successfully .
9
7
Second matrix is made successfully .
Product of matrices :
[66, 59]
[105, 97]
-----------------------------------------------------------
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 8 : - Write a program for sum of Diagonal and Non Diagonal elements .

Sumit_Ds_lab > Diagonal_Non_Diagonal.py > ...

```python
# Question 1 : Diagnol and Non - Diagonal sum :
print("------------Diagnol and Non - Diagonal sum -------")
#Take the size of the matrix from the user
n = int(input("Enter the matrix size : "))

#Make the matrix
a = []

#Take the elements of the matrix from the user
print("Enter the elements of the matrix:")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    a.append(row)

diagonal = 0
non_diagonal = 0

#Calculate the sum of diagonal and non-diagonal elements
for i in range(n):
    for j in range(n):
        if i == j:
            diagonal = diagonal + a[i][j]
        else:
            non_diagonal = non_diagonal + a[i][j]

print("The sum of diagonal elements:", diagonal)
print("The sum of non-diagonal elements:", non_diagonal)
print("------------------------------------------------------")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Diagonal_Non_Diagonal.py
------------Diagnol and Non - Diagonal sum -------
Enter the matrix size : 3
Enter the elements of the matrix:
2
3
4
5
6
7
8
9
2
The sum of diagonal elements: 10
The sum of non-diagonal elements: 36
------------------------------------------------------
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 9 : - Write a program for Stack Operation .

Sumit_Ds_lab > Stack_operation.py > ...

```python
# Stack operation

print("-------------- Stack operation -----------------")
stack = []

while True:
    print("1. for Push")
    print("2. for Pop")
    print("3. for Display")
    print("4. for Exit")

    choice = int(input("Enter the choice: "))

    if choice == 1:
        x = int(input("Enterthe element: "))
        stack.append(x)
        print("Element inserted:", x)

    elif choice == 2:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Popped element:", stack.pop())

    elif choice == 3:
        print("Stack is the = :", stack)

    elif choice == 4:
        break

    else:
        print("Invalid choice")


```

```python
# Reverse of string using stack

print("-------------- Reverse of string using stack -----------------")
s = input("Enter a string: ")
stack = []

for ch in s:
    stack.append(ch)

rev = ""

while len(stack) > 0:
    rev = rev + stack.pop()
print("Reverse string:", rev)
```

```python
# Matching Parentheses using stack

print("-------------- Matching Parentheses using stack -----------------")
s = input("Enter an expression: ")
stack = []
balanced = True

for ch in s:
    if ch == '(':
        stack.append(ch)

    elif ch == ')':
        if len(stack) == 0:
            balanced = False
            break
        else:
            stack.pop()

if len(stack) != 0:
    balanced = False
if balanced:
    print("Parentheses are balanced . ")
else:
    print("Parentheses are not balanced . ")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Stack_operation.py
-------------- Stack operation -----------------
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 1
Enterthe element: 45
Element inserted: 45
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 1
Enterthe element: 65
Element inserted: 65
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 2
```

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/Stack_operation.py
4. for Exit
Enter the choice: 1
Enterthe element: 65
Element inserted: 65
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 2
Popped element: 65
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 3
Stack is the = : [45]
1. for Push
2. for Pop
3. for Display
4. for Exit
Enter the choice: 4
-------------- Reverse of string using stack -----------------
```

```
-------------- Reverse of string using stack -----------------
Enter a string: Hello
Reverse string: olleH
-------------- Matching Parentheses using stack -----------------
Enter an expression: 89 % 3x * 9
Parentheses are balanced .
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Program 10 : - Write a program for general Queue .

```python
# General Queue Program
print("General Queue Program")
n = int(input("Enter queue size: "))

#Make the queue that is [0,0,0,...,0]
queue = [0] * n
front = -1
rear = -1

while True:
    print("Queue Operations:")
    print("1. for Enqueue")
    print("2. for Dequeue")
    print("3. for Display")
    print("4. for Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if rear == n - 1:
            print("Queue is full")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = rear + 1
            queue[rear] = x
            print("Element inserted:", x)
            print("-----------------------------------------------------------")

    elif choice == 2:
        if front == -1 or front > rear:
            print("Queue is empty . ")
        else:
            print("Deleted element:", queue[front])
            front = front + 1
```



```python
            print("-----------------------------------------------------------")

    elif choice == 2:
        if front == -1 or front > rear:
            print("Queue is empty . ")
        else:
            print("Deleted element:", queue[front])
            front = front + 1

            if front > rear:
                front = -1
                rear = -1
                print("Queue is now empty . ")
                print("-----------------------------------------------------------")

    elif choice == 3:
        if front == -1:
            print("Queue is empty")
        else:
            for i in range(front, rear + 1):
                print(queue[i], end=" ")
            print()
            print("-----------------------------------------------------------")

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

Output : -

```
neDrive/Desktop/Assigmnet/Sumit_Ds_lab/General_Queue.py
General Queue Program
Enter queue size: 4
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter your choice: 1
Enter element: 34
Element inserted: 34
------------------------------------------------------------
```

```
------------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter your choice: 2
Deleted element: 34
Queue is now empty .
------------------------------------------------------------
```

```
------------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter your choice: 4
```

Program 11 : - Write a program for Circular Queue.

```
print("------------Circular Queue Implementation -------")

#Take the size of the queue
n = int(input("Enter queue size : "))
#Make the queue that is [0,0,0,...,0]
queue = [0] * n
front = -1
rear = -1

```

Sumit_Ds_lab > Circular_queue.py > ...

```python
while True:
    print("Queue Operations:")
    print("1. for Enqueue")
    print("2. for Dequeue")
    print("3. for Display")
    print("4. for Exit")

    ch = int(input("Enter a choice: "))

    if ch == 1:
        if (rear + 1) % n == front:
            print("Queue is full")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = (rear + 1) % n
            queue[rear] = x
            print("Element inserted:", x)
            print("----------------------------------------------------------")

    elif ch == 2:
        if front == -1:
            print("Queue is an empty")
        else:
            print("Deleted element:", queue[front])

            if front == rear:
                front = -1
                rear = -1
            else:
                front = (front + 1) % n
                print("----------------------------------------------------------")
```

```python
                print("----------------------------------------------------------")

    elif ch == 3:
        if front == -1:
            print("Queue is empty")
        else:
            i = front
            while True:
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % n
            print()
            print("----------------------------------------------------------")

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output : -

```
------------Circular Queue Implementation -------
Enter queue size : 5
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter a choice: 1
Enter element: 56
Element inserted: 56
------------------------------------------------------
```

```
------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter a choice: 1
Enter element: 59
Element inserted: 59
------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter a choice: 2
Deleted element: 56
```

```
------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter a choice: 3
59
------------------------------------------------------
Queue Operations:
1. for Enqueue
2. for Dequeue
3. for Display
4. for Exit
Enter a choice: 4
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```

Assignment Summary & Key Learnings

This practical assignment provided hands-on experience in implementing fundamental Data Structures and algorithmic logic using Python. Through the completion of these 12 practical programs, several core computer science concepts were thoroughly explored and reinforced:

1. Linear & Non-Linear Data Structures: Practiced dynamic array operations (insertion and deletion), implemented Last-In-First-Out (LIFO) stacks for string reversal and expression validation (parentheses matching), and developed First-In-First-Out (FIFO) general queues as well as memory-efficient circular queues.

2. Matrix Computations: Gained proficiency in 2D list manipulations, calculating principal diagonal sums, anti-diagonal non-diagonal sums, and performing complex matrix multiplication.

3. Core Programming Logic & Conditional Statements: Enhanced problem-solving abilities by building practical utility scripts including slab-based electricity billing systems, even/odd number checkers, largest/smallest number finders, and percentage-based student grading systems.

Overall, this assignment bridged theoretical data structures concepts with practical Python programming, strengthening analytical skills essential for Master of Computer Applications (MCA) coursework and future software development.

THANK YOU

Data Structures Using Python • MCA 1st Semester
Manav Rachna International Institute of Research & Studies